



De l'énoncé au code

4PYTH – Correction guidée du défi 1

Chris CHEVALIER



La méthode en 4 étapes

1. Comprendre

Qu'est-ce qui entre dans la fonction ?
Qu'est-ce qui en sort ? Que donnent les exemples de l'énoncé ?

2. Découper

Écrire les étapes **en français**, une par ligne, avant de taper la moindre ligne de code. C'est le **pseudo-code**.

3. Coder petit à petit

Une étape du plan à la fois. Le programme doit tourner après chaque ajout, même s'il ne fait pas encore tout.

4. Tester tout de suite

Relancer avec les exemples de l'énoncé après chaque étape. Une erreur trouvée tôt est une erreur facile à corriger.



Étape 1 – Comprendre l'énoncé

Question	Réponse
Entrées	password (une chaîne de caractères) et is_admin (True ou False , False par défaut)
Sortie	un tuple : True ou False , puis la liste des messages d'erreur
Exemple valide	check_password("Tr0ub4dour&Co") renvoie (True, [])
Exemple refusé	check_password("azerty") renvoie False et 5 messages. Il ne respecte presque aucune règle

Le mot de passe est valide **si et seulement si la liste des erreurs est vide**. Il suffit de remplir cette liste, règle après règle.



Étape 2 – Le plan en français

```
Pseudo-code

créer une liste d'erreurs vide
si le mot de passe est trop court : ajouter une erreur
regarder chaque caractère du mot de passe :
    noter les catégories vues : majuscule, minuscule, chiffre, spécial
pour chaque catégorie jamais rencontrée : ajouter une erreur
si le mot de passe est un mot de passe courant : ajouter une erreur
renvoyer (la liste est-elle vide ?, la liste des erreurs)
```

Chaque ligne du plan deviendra quelques lignes de Python. Le plus dur est fait. Il ne reste qu'à traduire.



Étape 3a – Le squelette

```
challenge1.py

def check_password(password, is_admin=False):
    """Vérifie un mot de passe."""
    errors = []

    # ... les règles viendront ici ...

    return len(errors) == 0, errors

print(check_password("azerty"))  # (True, [])
```

- La fonction **tourne déjà**. Elle accepte tout, mais elle renvoie le bon type de résultat
- `len(errors) == 0` vaut `True` quand la liste est vide
- `return a, b` renvoie un **tuple**



Étape 3b – La longueur

```
challenge1.py

MIN_LENGTH = 12
ADMIN_MIN_LENGTH = 16

def check_password(password, is_admin=False):
    errors = []

    if is_admin:
        min_length = ADMIN_MIN_LENGTH
    else:
        min_length = MIN_LENGTH
    if len(password) < min_length:
        errors.append(f"au moins {min_length} caractères")

    return len(errors) == 0, errors
```

- Les seuils sont des **constantes** en majuscules. On les modifie à un seul endroit
- Test :
`check_password("azerty")`
renvoie maintenant `(False, ['au moins 12 caractères'])`



Étape 3c – Regarder chaque caractère

```
challenge1.py (suite)

has_upper = False
has_lower = False
has_digit = False
has_special = False
for char in password:
    if char.isupper():
        has_upper = True
    elif char.islower():
        has_lower = True
    elif char.isdigit():
        has_digit = True
    elif not char.isalnum():
        has_special = True
```

- Quatre **drapeaux** (des booléens) partent à `False` : « je n'ai encore rien vu »
- La boucle les passe à `True` dès qu'elle rencontre la catégorie
- `isalnum()` est vrai pour une lettre ou un chiffre. `not char.isalnum()` repère donc un caractère spécial



Étape 3d – Conclure puis vérifier la liste

```
challenge1.py (suite)

if not has_upper:
    errors.append("au moins une majuscule")
if not has_lower:
    errors.append("au moins une minuscule")
if not has_digit:
    errors.append("au moins un chiffre")
if not has_special:
    errors.append("au moins un caractère spécial")

common = [p.lower() for p in COMMON_PASSWORDS]
if password.lower() in common:
    errors.append("ne doit pas être un mot de passe courant")

return len(errors) == 0, errors
```

- Des `if` **indépendants** (et non des `elif`). Plusieurs règles peuvent échouer en même temps
- On compare tout **en minuscules**. « AZERTY » est aussi faible que « azerty »
- Le `return` reste **à la fin**, après toutes les vérifications



Étape 4 – Tester avec les exemples

```
Tests

print(check_password("Tr0ub4dour&Co"))
# (True, [])
print(check_password("Azerty123456!"))
# (False, ['ne doit pas être un mot de passe courant'])
print(check_password("azerty"))
# (False, ['au moins 12 caractères', 'au moins une majuscule', 'au moins un chiffre',
#         'au moins un caractère spécial', 'ne doit pas être un mot de passe courant'])
print(check_password("Tr0ub4dour&Co", is_admin=True))
# (False, ['au moins 16 caractères'])
```

Si un résultat diffère, on relit l'étape correspondante du plan. C'est là que se trouve l'erreur.



Les pièges les plus fréquents

Piège	Correction
<code>return</code> placé dans la boucle	la fonction s'arrête au premier caractère : le <code>return</code> va à la fin
une chaîne d' <code>elif</code> pour les règles	une seule erreur est signalée : des <code>if</code> indépendants
comparer sans <code>lower()</code>	« AZERTY » passe le contrôle : comparer en minuscules
renvoyer seulement <code>True</code> ou <code>False</code>	l'énoncé demande aussi la liste : <code>return valide, errors</code>
tout écrire d'un coup puis tester	coder et tester étape par étape

Cette méthode s'applique à **chaque exercice** : comprendre, découper en français, coder petit à petit, tester.



Des questions ?

Chris CHEVALIER

chris.chevalier@ecole-hexagone.com